# 임베디드 시스템

## Embedded Systems

LCD

Potentiometer

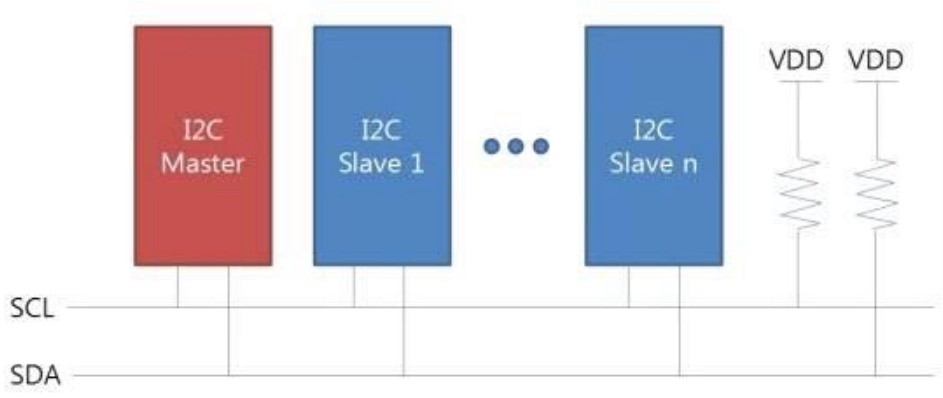
CDS

강원대학교 컴퓨터공학과 권용훈

# I2C(1/4)

## I2C 통신

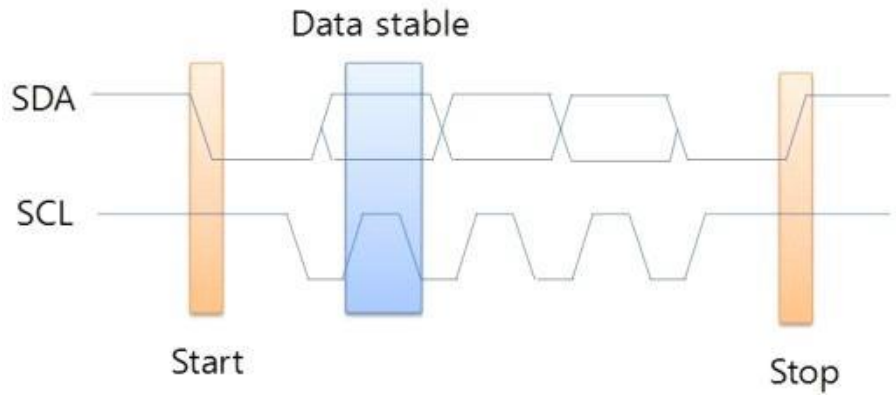
- I2C(Inter-Integrated Circuit): 두 개의 버스를 이용한 직렬 데이터 통신 방식
- Master와 Slave로 구분되며, Master와 Slave 간에 1 : n 연결이 가능하다.




- SCL(Serial Clock): 클록을 통한 동기화를 위한 핀
- SDA(Serial Data): 데이터를 주고 받기 위한 핀
- Address: slave에 주어지는 고유한 주소 값

# I2C(2/4)

## I2C 통신 방식




- 두 핀 모두 기본 값은 HIGH 이며, SDA가 LOW 로 떨어질 때 시작 신호로 판단하여 통신이 시작된다.
- 통신할 Slave 주소를 송신하여 해당 주소의 Slave와 통신을 수행
- SCL이 LOW 일 때, 데이터를 비트로 변환한다.
- SCL이 HIGH 일 때, 데이터를 읽어온다.
- Master는 두 핀 모두 HIGH 일 때, SDA를 HIGH로 변경하여 통신을 종료한다.

# I2C(3/4)

## I2C 통신의 장단점

- 장점
  - 2개의 입출력 핀과 소프트웨어를 이용하여 여러 장치들과 Master & Slave 사이의 반 이중 통신 제어
  - 시스템 동작 중 일 때도 I2C 버스에 추가 제거가 가능하다.
  - SCL 핀으로 클록을 맞추어서 동작하는 동기방식으로 안정적이다.
  - 하드웨어 구성이 간단하며 대화형 동작 가능하다.
  - 멀티 마스터 설정 가능
  - 칩 생산 비용과 설계비용, 기기의 전력 소모 감소
- 단점
  - 최대 100kHz 또는 고속 최대 400kHz까지의 다른 통신에 비교적 저속의 데이터 전송
  - 저속 주변 장치와의 데이터 통신 및 주변 장치의 동작 모드 설정에 적합
  - 동시 양방향 통신 불가
  - 7bit 주소 길이 제한으로 최대 128개 Slave 장치 주소 충돌 가능성 높음

# I2C(4/4)

## I2C 핀맵

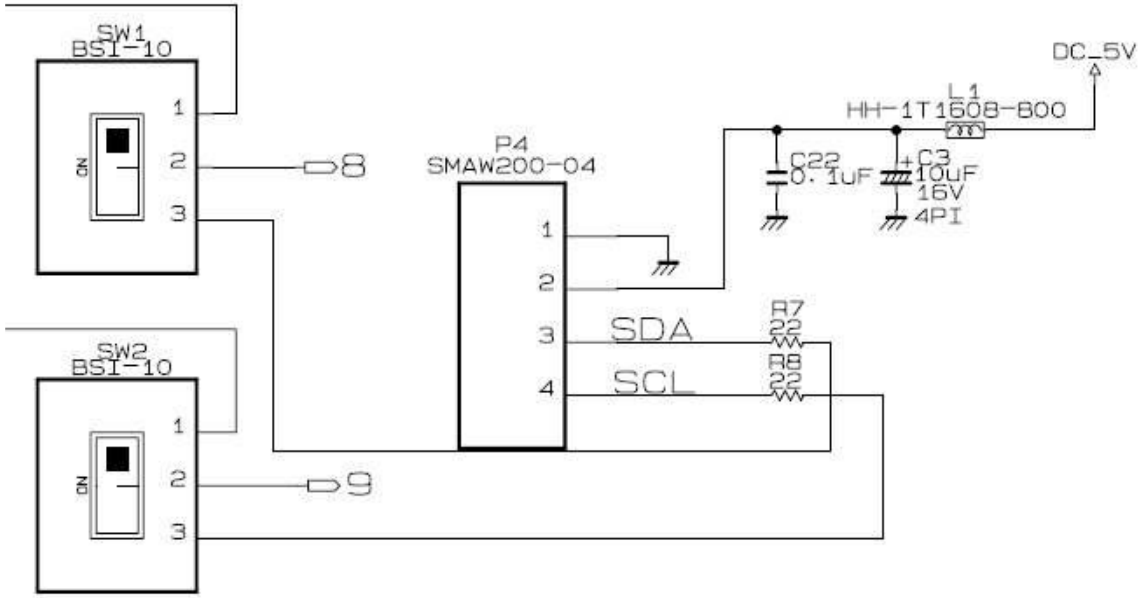
| BCM | wPi | Name | Mode | V | Physical | V | Mode | Name | wPi | BCM |
|---|---|---|---|---|---|---|---|---|---|---|
| | | 3.3v | | | 1 \|\| 2 | | | 5v | | |
| 2 | 8 | SDA.1 | ALT0 | 1 | 3 \|\| 4 | | | 5v | | |
| 3 | 9 | SCL.1 | ALT0 | 1 | 5 \|\| 6 | | | 0v | | |
| 4 | 7 | GPIO. 7 | IN | 1 | 7 \|\| 8 | 0 | IN | TxD | 15 | 14 |
| | | 0v | | | 9 \|\| 10 | 1 | IN | RxD | 16 | 15 |
| 17 | 0 | GPIO. 0 | IN | 1 | 11 \|\| 12 | 1 | IN | GPIO. 1 | 1 | 18 |
| 27 | 2 | GPIO. 2 | IN | 0 | 13 \|\| 14 | | | 0v | | |
| 22 | 3 | GPIO. 3 | IN | 1 | 15 \|\| 16 | 0 | OUT | GPIO. 4 | 4 | 23 |
| | | 3.3v | | | 17 \|\| 18 | 0 | IN | GPIO. 5 | 5 | 24 |
| 10 | 12 | MOSI | ALT0 | 0 | 19 \|\| 20 | | | 0v | | |
| 9 | 13 | MISO | ALT0 | 0 | 21 \|\| 22 | 0 | IN | GPIO. 6 | 6 | 25 |
| 11 | 14 | SCLK | ALT0 | 0 | 23 \|\| 24 | 1 | OUT | CE0 | 10 | 8 |
| | | 0v | | | 25 \|\| 26 | 1 | OUT | CE1 | 11 | 7 |
| 0 | 30 | SDA.0 | IN | 0 | 27 \|\| 28 | 0 | IN | SCL.0 | 31 | 1 |
| 5 | 21 | GPIO.21 | OUT | 0 | 29 \|\| 30 | | | 0v | | |
| 6 | 22 | GPIO.22 | IN | 0 | 31 \|\| 32 | 0 | IN | GPIO.26 | 26 | 12 |
| 13 | 23 | GPIO.23 | IN | 0 | 33 \|\| 34 | | | 0v | | |
| 19 | 24 | GPIO.24 | IN | 0 | 35 \|\| 36 | 0 | OUT | GPIO.27 | 27 | 16 |
| 26 | 25 | GPIO.25 | IN | 0 | 37 \|\| 38 | 0 | IN | GPIO.28 | 28 | 20 |
| | | 0v | | | 39 \|\| 40 | 0 | IN | GPIO.29 | 29 | 21 |

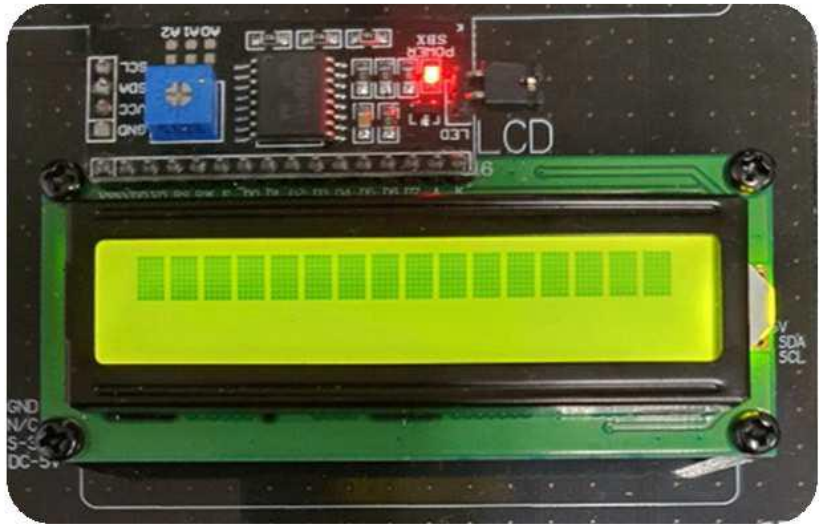
| 연결핀 | 기능 | Raspberry Pi 핀 배열 |
|---|---|---|
| SCL0 | 클럭 | SPI_SCLK |
| SDA0 | 데이터 송수신 핀 | SPI_MOSI |
| SCL1 | 클럭 | SPI_MISO |
| SDA1 | 데이터 송수신 핀 | SPI_CEn |

# LCD(1/8)

## LCD (Liquid Crystal Display)

- ASCII 문자를 표현할 수 있는 장치




| BCM | wPi | Name | Mode | V | Physical | | V | Mode | Name | wPi | BCM |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 3.3v | | | 1 | 2 | | | 5v | | |
| 2 | 8 | SDA.1 | ALT0 | 1 | 3 | 4 | | | 5v | | |
| 3 | 9 | SCL.1 | ALT0 | 1 | 5 | 6 | | | 0v | | |
| 4 | 7 | GPIO. 7 | IN | 1 | 7 | 8 | 0 | IN | TxD | 15 | 14 |
| | | 0v | | | 9 | 10 | 1 | IN | RxD | 16 | 15 |

# LCD(2/8)

## Text LCD 모듈의 제어 명령

fosc = 270kHz

| 기능 | 제어신호 | | 제어명령어 | | | | | | | 실행시간 |
|---|---|---|---|---|---|---|---|---|---|---|
| | Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 | (Max.) |
| Clear Display | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1.52ms |
| Return Home | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | | 1.52ms |
| Entry Mode Set | 0 | 0 | 0 | 0 | 0 | 0 | 1 | I/D | S | 37us |
| Display On/Off Control | 0 | 0 | 0 | 0 | 0 | 1 | D | C | B | 37us |
| Cursor or Display Shift | 0 | 0 | 0 | 0 | 1 | S/C | R/L | | | 37us |
| Function Set | 0 | 0 | 0 | 1 | DL | N | F | | | 37us |
| Set CG RAM Address | 0 | 0 | 0 | 1 | CG RAM Address | | | | | 37us |
| Set DD RAM Address | 0 | 0 | 1 | DD RAM Address | | | | | | 37us |
| Read busy flag and address | 0 | 1 | BF | Address Count | | | | | | 0us |
| Data Write to CGRAM or DDRAM | 1 | 0 | Write Data | | | | | | | 41us |
| Data Read from CGRAM or DDRAM | 1 | 1 | Read Data | | | | | | | 41us |

**(1) Clear Display : 0b0000 0001 (0x01)**

전체 화면을 지우고 DDRAM 주소를 0으로 설정한 후 커서를 홈 위치로 옮기면 DDRAM 어드레스 카운트를 0으로 한다.

데이터 버스에는 0000 0001 (0x01)이 전송된다.

| Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

**(2) Return Home : 0b0000001x (0x02)**

커서를 홈 위치로 옮긴다. DDRAM 주소 카운터를 0으로 하고 디스플레이 시프트된 것을 본래의 위치로 되돌린다.

| Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | X |

**(3) Entry Mode Set**

MCU가 LCD 모듈에 데이터를 Read/Write할 경우에 DDRAM 어드레스 즉 커서의 위치를 오른쪽으로 증가할 것인지 왼쪽으로 감소할 것인지 결정하며, 화면을 시프트 할 것인지 설정한다.

| Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | I/D | S |

I/D => Increment/Decrement S => Shift

| I/D | S | Value | 기능 |
|---|---|---|---|
| 0 | 0 | 0b 0000 0100 (0x04) | 커서(어드레스) 좌측 이동, 화면 이동 없음 |
| 0 | 1 | 0b 0000 0101 (0x05) | 커서(어드레스) 좌측 이동, 화면 이동 |
| 1 | 0 | 0b 0000 0110 (0x06) | 커서(어드레스) 우측 이동, 화면 이동 없음 |
| 1 | 1 | 0b 0000 0111 (0x07) | 커서(어드레스) 우측 이동, 화면 이동 |

# LCD(3/8)

### (4) Display On/Off

현재 DD RAM의 내용이 화면에 표시되도록 On/Off 설정하거나, 커서(어드레스)를 On/Off하거나, 커서 위치의 문자를 깜빡이게 할 것인지 설정한다.

| Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | D | C | B |

D => Display C => Cursor(Address) B => Blink

| D | C | V | Value | 기능 | | |
|---|---|---|---|---|---|---|
| | | | | 화면 | 커서 | 커서 점멸 |
| 0 | 0 | 0 | 0b 0000 1000 (0x08) | OFF | OFF | OFF |
| 0 | 0 | 1 | 0b 0000 1001 (0x09) | OFF | OFF | ON |
| 0 | 1 | 0 | 0b 0000 1010 (0x0A) | OFF | ON | OFF |
| 0 | 1 | 1 | 0b 0000 1011 (0x0B) | OFF | ON | ON |
| 1 | 0 | 0 | 0b 0000 1100 (0x0C) | ON | OFF | OFF |
| 1 | 0 | 1 | 0b 0000 1101 (0x0D) | ON | OFF | ON |
| 1 | 1 | 0 | 0b 0000 1110 (0x0E) | ON | ON | OFF |
| 1 | 1 | 1 | 0b 0000 1111 (0x0F) | ON | ON | ON |

### (5) Cursor Or Display Shift

DD RAM의 내용을 변경하지 않고 화면 또는 커서(어드레스)를 오른쪽 또는 왼쪽으로 시프트(shfit)한다.

| Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | S/C | R/L | X | X |

S/C => Shift/Cursor R/L => Right/Left

| S/C | R/L | Value | 기능 |
|---|---|---|---|
| 0 | 0 | 0b 0001 00XX (0x10) | 커서(어드레스) 선택, 커서(어드레스) 좌측 이동 |
| 0 | 1 | 0b 0001 01XX (0x14) | 커서(어드레스) 선택, 커서(어드레스) 우측 이동 |
| 1 | 0 | 0b 0001 10XX (0x18) | 화면 선택, 화면 좌측 이동 |
| 1 | 1 | 0b 0001 11XX (0x1C) | 화면 선택, 화면 우측 이동 |

### (6) Function Set

MCU와 LCD 모듈 사이에 인터페이스 설정을 한다.

데이터 길이, 화면 표시 행수, 문자 폰트 크기를 설정한다.

| Rs | R/nW | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | DL | N | F | X | X |

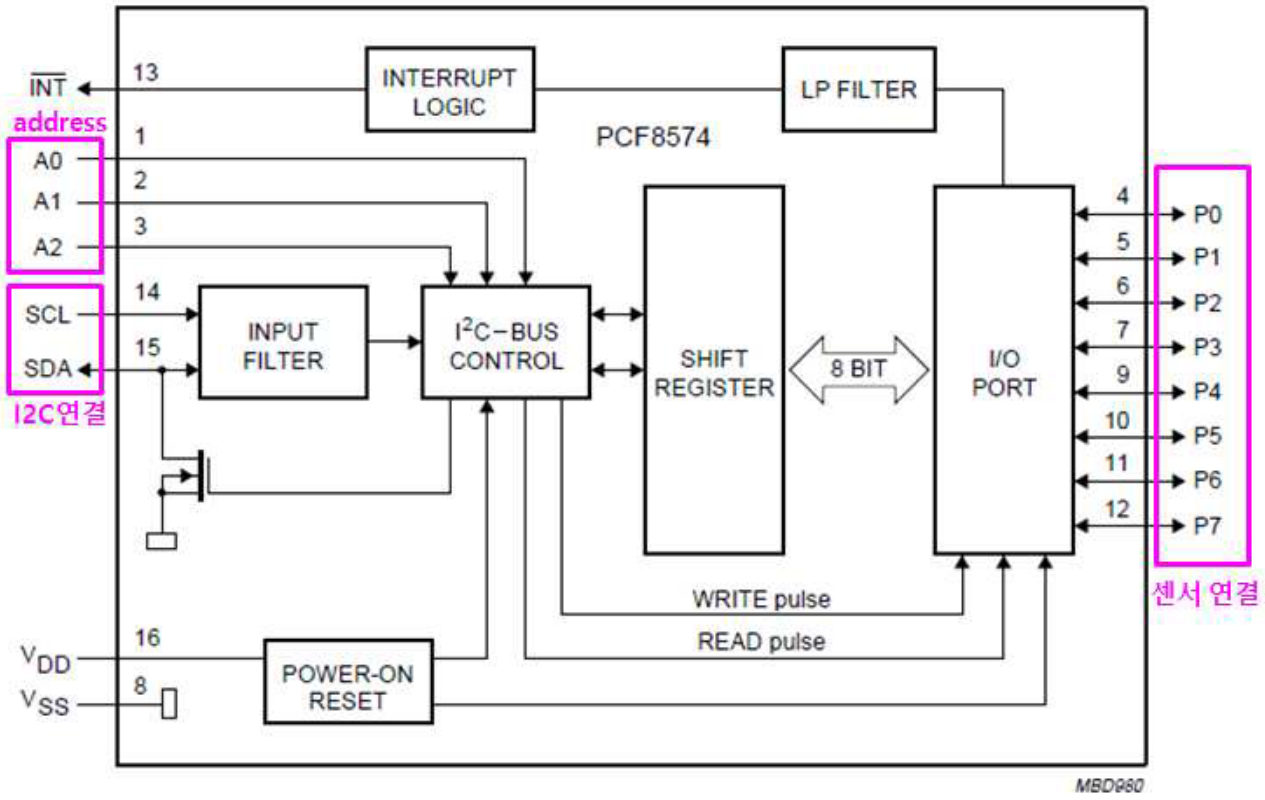
DL => Data Length N => New Line F => Font

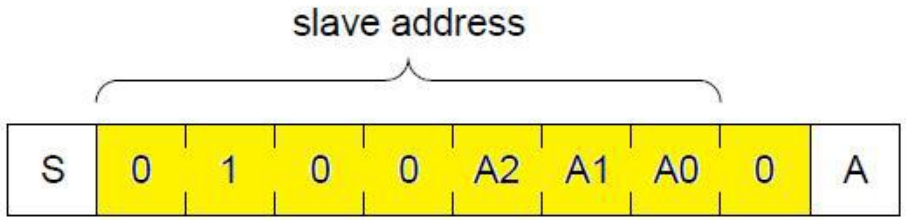
| DL | N | F | 기능 | | |
|---|---|---|---|---|---|
| | | | 데이터 길이 | 행 수 | 폰트 크기 |
| 0 | 0 | 0 | 4bit | 1행 | 5 * 8 |
| 0 | 0 | 1 | 4bit | 1행 | 5 * 11 |
| 0 | 1 | 0 | 4bit | 2행 | 5 * 8 |
| 0 | 1 | 1 | 4bit | 2행 | 5 * 11 |
| 1 | 0 | 0 | 8bit | 1행 | 5 * 8 |
| 1 | 0 | 1 | 8bit | 1행 | 5 * 11 |
| 1 | 1 | 0 | 8bit | 2행 | 5 * 8 |
| 1 | 1 | 1 | 8bit | 2행 | 5 * 11 |

# LCD(4/8)

## PCF8574 IC

- 2개의 핀을 사용하는 I2C 방식의 집적회로




| Pin Connectivity | | | Address of PCF8547 | | | | | | | | 7bit |
|---|---|---|---|---|---|---|---|---|---|---|---|
| A2 | A1 | A0 | A6 | A5 | A4 | A3 | A2 | A1 | A0 | 0 | Address |
| L | L | L | 0 | 1 | 0 | 0 | 0 | 0 | 0 | - | 20h |
| L | L | H | 0 | 1 | 0 | 0 | 0 | 0 | 1 | - | 21h |
| L | H | L | 0 | 1 | 0 | 0 | 0 | 1 | 0 | - | 22h |
| L | H | H | 0 | 1 | 0 | 0 | 0 | 1 | 1 | - | 23h |
| H | L | L | 0 | 1 | 0 | 0 | 1 | 0 | 0 | - | 24h |
| H | L | H | 0 | 1 | 0 | 0 | 1 | 0 | 1 | - | 25h |
| H | H | L | 0 | 1 | 0 | 0 | 1 | 1 | 0 | - | 26h |
| H | H | H | 0 | 1 | 0 | 0 | 1 | 1 | 1 | - | 27h |

# LCD(5/8)

## 라이브러리 함수

| int lcdInit ( const int rows, const int cols, const int bits, const int rs, const int strb, const int d0, const int d1, const int d2, const int d3, const int d4, const int d5, const int d6, const int d7) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | I2C 통신할 슬레이브를 식별할 주소 설정 | |
| 매개변수 | const int rows | LCD의 라인 수 |
| | const int cols | LCD의 칸 수 |
| | const int bits | LCD의 데이터 제어 비트 수 ( 8bit / 4bit ) |
| | const int rs | 데이터 레지스터, 명령어 레지스터 선택 핀 |
| | const int strb | LCD에 데이터를 쓰겠다는 신호 핀 |
| | const int d0 | 데이터 핀0 |
| | const int d1 | 데이터 핀1 |
| | const int d2 | 데이터 핀2 |
| | const int d3 | 데이터 핀3 |
| | const int d4 | 데이터 핀4 |
| | const int d5 | 데이터 핀5 |
| | const int d6 | 데이터 핀6 |
| | const int d7 | 데이터 핀7 |
| 리턴 값 | int | |

| int pcf8574Setup(const int pinBase, const int i2cAddress) | | |
|---|---|---|
| 위 치 | wiringPi/wiringPi/pcf8574.h | |
| 기 능 | PCF8574 IC와 LCD 연결 설정 및 초기화 | |
| 매개변수 | const int pinBase | 64이상의 8개 핀의 시작 주소 |
| 리턴 값 | const int i2cAddress | I2C통신 주소 |

| void lcdHome(const int fd) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | 커서를 0번 칸, 0번 줄로 이동 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| 리턴 값 | void | 없음 |

| void lcdClear(const int fd) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD화면 지우기 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| 리턴 값 | void | 없음 |

# LCD(6/8)

## 라이브러리 함수

| void lcdDisplay(const int fd, int state) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD를 화면 On/Off 여부 설정 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | int state | 화면 On/Off 여부 |
| 리턴 값 | void | 없음 |

| void lcdCursor(const int fd, int state) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD에 커서 표시 여부 설정 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | int state | 커서 표시 여부 |
| 리턴 값 | void | 없음 |

| void lcdCursorBlink(const int fd, int state) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD에 커서 깜빡임 여부 설정 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | int state | 커서 깜빡임 여부 |
| 리턴 값 | void | 없음 |

| void lcdSendCommand(const int fd, unsigned char cmd) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD에 명령어 전송 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | unsigned char cmd | LCD에 보낼 커맨드 |
| 리턴 값 | void | 없음 |

| void lcdPosition(const int fd, int x, int y) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | 지정된 위치에 커서 위치 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | int x | 커서가 놓일 x좌표 |
| | int y | 커서가 놓일 y좌료 |
| 리턴 값 | void | 없음 |

| void lcdPutchar(const int fd, unsigned char data) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD에 문자 출력 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | unsigned char data | 출력할 ASCII 문자 |
| 리턴 값 | void | 없음 |

# LCD(7/8)

## 라이브러리 함수

| void lcdPuts(const int fd, const char *string) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD에 문자열 출력 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | const char *string | 출력할 ASCII 문자열 |
| 리턴 값 | void | 없음 |

| void lcdPrintf(const int fd, const char *message, . . .) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | LCD에 형식지정 문자열 출력 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | const char *message | 출력할 형식지정 문자열 |
| 리턴 값 | void | 없음 |

| void lcdCharDef(const int fd, int x, unsigned char data[8]) | | |
|---|---|---|
| 위 치 | wiringPi/devLib/lcd.h | |
| 기 능 | 8개 문자를 사용자 정의 | |
| 매개변수 | const int fd | LCD 커서 제어 핸들 |
| | int index | 커서가 놓일 x좌표 |
| | unsigned char data[8] | 사용자 정의 8개 문자 |
| 리턴 값 | void | 없음 |

```
pi@raspberrypi:~/Example/I2C $ gcc -lwiringPi -lwiringPiDev EX_I2CLCD.c -o LCD
pi@raspberrypi:~/Example/I2C $ ls
EX_I2CLCD.c  LCD
```

# LCD(8/8)

[ EX_I2C_LCD.c ]

```c
//- 관련 라이브러리 추가 ----------------------------------------------------------
#include <wiringPi.h>
#include <pcf8574.h>
#include <lcd.h>
#include <stdio.h>

//- PCF8574 &#8212; LCD 연결핀 주소값 선언 ----------------------------------------
#define BASE_ADDR          100
#define RS_PIN             BASE_ADDR          //- 100
#define RW_PIN             BASE_ADDR+1        //- 101
#define EN_PIN             BASE_ADDR+2        //- 102
#define BL_PIN             BASE_ADDR+3        //- 103
#define D4_PIN             BASE_ADDR+4        //- 104
#define D5_PIN             BASE_ADDR+5        //- 105
#define D6_PIN             BASE_ADDR+6        //- 106
#define D7_PIN             BASE_ADDR+7        //- 107

//- 기능 구현 ----------------------------------------------------------------------
int main (void)
{
    //- wiringPi 초기화
    if( wiringPiSetup() == -1)
    {
        printf("Setup Fail ~!!");
        return -1;
    }
```

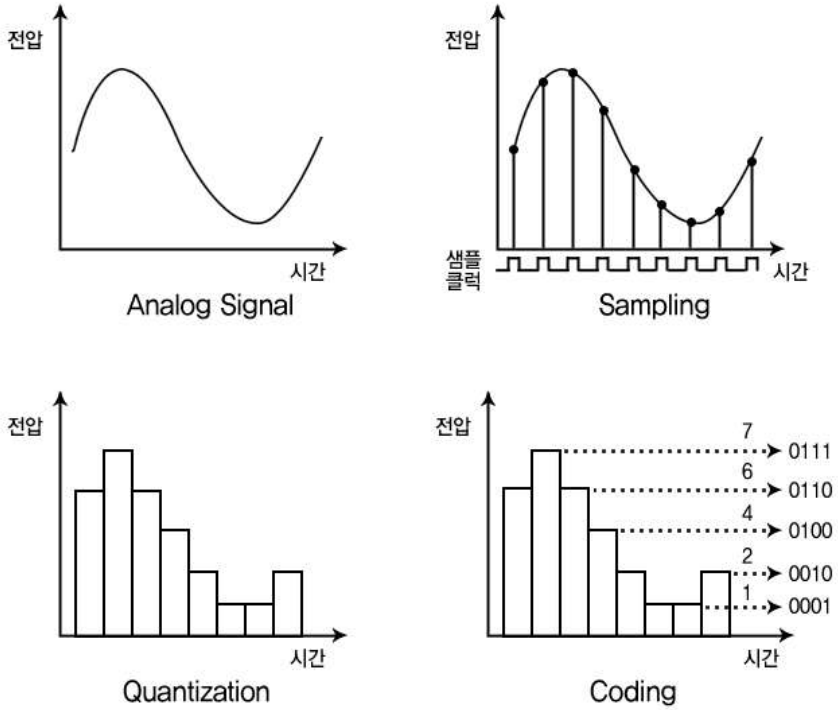
[ EX_I2C_LCD.c ]

```c
    //- I2C 통신기반 LCD 슬레이브 설정
    pcf8574Setup(RS_PIN, 0x27);        //- P0핀 - LCD의 RS 연결, 주소 지정

    //- P1핀 - LCD의 RW 연결, 동작모드 설정
    pinMode (RW_PIN, OUTPUT);
    digitalWrite (RW_PIN, LOW);

    //- P3핀 - LCD의 Backlight 연결
    pinMode (BL_PIN, OUTPUT);
    digitalWrite (BL_PIN, HIGH);       //- Backlight ON


    //- LCD 제어 핸들
    int fd = lcdInit (2, 16, 4, RS_PIN,  EN_PIN,  D4_PIN, D5_PIN, D6_PIN,
                      D7_PIN, 0,0,0,0 );

    while(1)
    {
        lcdClear(fd);                  //- LCD화면 지우기
        delay(500);

        lcdPosition (fd, 2, 0);        //- 0번 칸, 0번 줄 커서 위치
        lcdPuts (fd, "Hello, World!");  //- 문자열 출력
        lcdPosition (fd, 2, 1);        //- 0번 칸, 1번 줄 커서 위치
        lcdPuts (fd, "* Good Luck *");  //- 문자열 출력
        delay(2000);

    }
    lcdClear(fd);                      //- LCD화면 지우기
    delay(500);

    return 0;
}
```

# Potentiometer(1/5)

## ADC(Analog-to-Digital Converter)

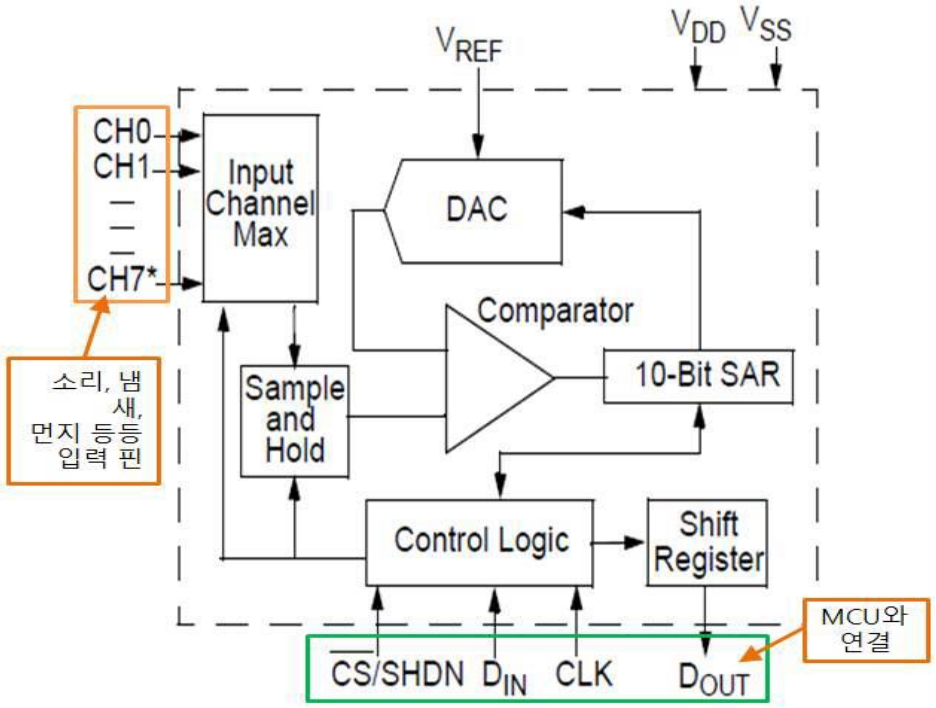
- 연속적인 값을 가지는 아날로그 신호를 디지털 신호로 변환하는 장치
- 다음과 같은 절차를 통해 변환된다.
- 표본화(Sampling)
  - 아날로그 입력 시 일정 시간 간격으로 표본을 추출한 것
  - 일정 시간 간격이 짧을수록 아날로그 입력 신호에 근접한 모양의 표본 추출 가능
  - 단위 : SPS (Sample Per Second / Sampling Rate)
- 양자화(Quantization)
  - 시간에 따른 신호의 세기를 이산화 시켜 표시하는 것
  - 이산화 값의 범위가 클수록 정밀도가 높으며 이것을 해상도(Resolution)이라 한다.
  - 8bit ( 0~ 255), 10bit ( 0 ~ 1023 )
- 부호화(Coding)
  - 표본화 된 데이터를 디지털 값으로 맵핑시켜 사용하는 것

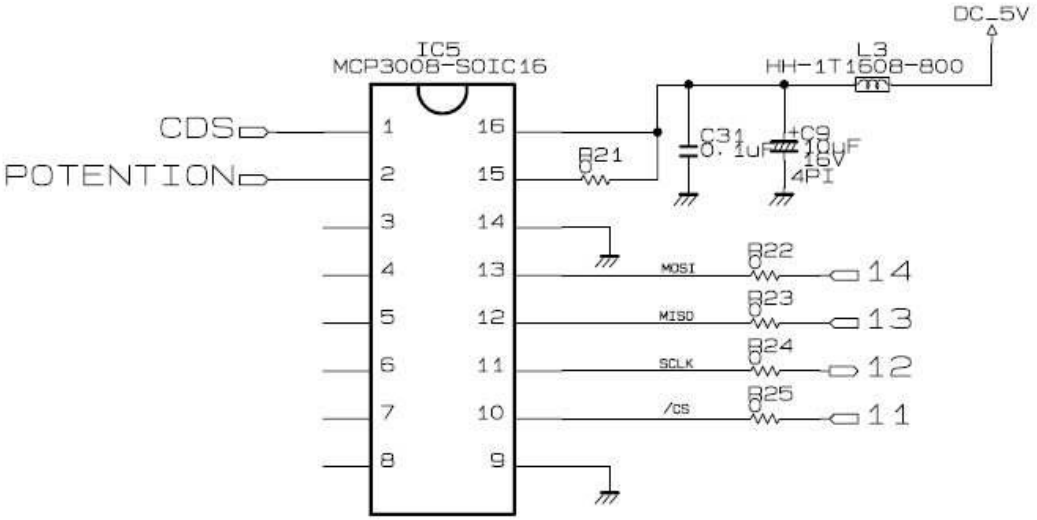


# Potentiometer(2/5)

## MCP3008 IC

- 아날로그 데이터를 SPI방식으로 입력 받는 집적회로

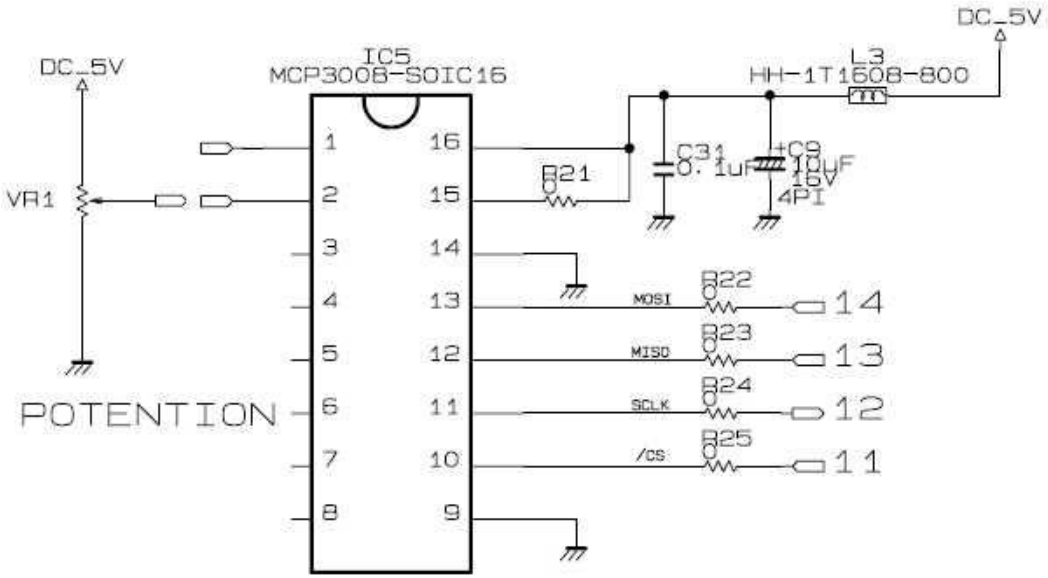


# Potentiometer(3/5)

## 가변저항(Potentiometer)

- 전자회로의 저항 값을 정해진 범위 내에서 임의로 바꿀 수 있는 저항



| BCM | wPi | Name | Mode | V | Physical | V | Mode | Name | wPi | BCM |
|---|---|---|---|---|---|---|---|---|---|---|
| | | 3.3v | | | 17 \|\| 18 | 0 | IN | GPIO. 5 | 5 | 24 |
| 10 | 12 | MOSI | ALT0 | 0 | 19 \|\| 20 | | | 0v | | |
| 9 | 13 | MISO | ALT0 | 0 | 21 \|\| 22 | 0 | IN | GPIO. 6 | 6 | 25 |
| 11 | 14 | SCLK | ALT0 | 0 | 23 \|\| 24 | 1 | OUT | CE0 | 10 | 8 |
| | | 0v | | | 25 \|\| 26 | 1 | OUT | CE1 | 11 | 7 |
| 0 | 30 | SDA.0 | IN | 0 | 27 \|\| 28 | 0 | IN | SCL.0 | 31 | 1 |
| 5 | 21 | GPIO.21 | OUT | 0 | 29 \|\| 30 | | | 0v | | |
| 6 | 22 | GPIO.22 | IN | 0 | 31 \|\| 32 | 0 | IN | GPIO.26 | 26 | 12 |
| 13 | 23 | GPIO.23 | IN | 0 | 33 \|\| 34 | | | 0v | | |
| 19 | 24 | GPIO.24 | IN | 0 | 35 \|\| 36 | 0 | OUT | GPIO.27 | 27 | 16 |
| 26 | 25 | GPIO.25 | IN | 0 | 37 \|\| 38 | 0 | IN | GPIO.28 | 28 | 20 |
| | | 0v | | | 39 \|\| 40 | 0 | IN | GPIO.29 | 29 | 21 |

Pi 3B+

# Potentiometer(4/5)

[ EX_Potentionmeter.c ]

```c
//- 센서 제어 라이브러리 ----------------------------------------------------------
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>
#include <errno.h>
#include <string.h>

//- 센서 제어 상수 선언 ----------------------------------------------------------
#define CHAN_CONFIG_SINGLE        8          //- 싱글 채널 설정 값
#define SPI_SPEED                 1000000    //- 1MHz
#define SPI_CE_CHANNEL            1          //- SPI와 연결된 CE 채널
#define SPI_ADC_POTEN_CHANNEL      1         //- Potentionmeter 연결 채널
#define CS_MCP3208                11         //- MCP3208 연결 핀

//- 함수선언----------------------------------------------------------------------
int spiSetup(void);
int readAnalogData(int adcChannel);

//- 프로그램 실행 함수 ------------------------------------------------------------
int main(void){

    //- wiringPi 초기화
    if( wiringPiSetup() == -1)
    {
            printf("Setup Fail ~!!");
            return -1;
    }
```

[ EX_Potentionmeter.c ]

```c
    //- SPI통신 초기화
    spiSetup();

    //- ADC IC MCP3008 초기화
    pinMode(CS_MCP3208, OUTPUT);

    int adcValue = 0;

    //- 센서 읽어서 출력
    while(1)
    {
        //- 센서 값 읽기
        adcValue = readAnalogData(SPI_ADC_POTEN_CHANNEL);
        //- 입력된 가변저항 값 출력하기
        printf("  Potentionmeter Value = %d₩n", adcValue);
        delay(500);
    }
    return 0;
}


//- SPI통신 라이브러리 초기화------------------------------------------------------
int spiSetup(void)
{
    if( wiringPiSPISetup (SPI_CE_CHANNEL, SPI_SPEED) == -1 )
    {
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s₩n", strerror (errno));
        return 1;
    }
}
```

# Potentiometer(5/5)

[ EX_Potentionmeter.c ]

```c
//- 채널 1번에서 측정된 ADC센서 읽어오기 ----------------------------------------
int readAnalogData(int adcChannel)
{
    int adcValue = 0;

    unsigned char buffer[3] = {0};

    //- MCP3008의 8개 채널 중 하나인지 체크
    if(adcChannel<0 || adcChannel>7)  return -1;

    //- SPI 통신 버퍼 설정
    buffer[0] = 0x01;

    buffer[1] = (CHAN_CONFIG_SINGLE+adcChannel) << 4;


    //- SPI 통신할 칩 활성화
    digitalWrite(CS_MCP3208, LOW);


    //- 채널 1에서 측정된 ADC 센서 읽어 오기
    wiringPiSPIDataRW(SPI_CE_CHANNEL, buffer, 3);

    //- 읽어온 값에서 10비트 데이터 추출하기
    adcValue = ( (buffer[1] & 3 ) << 8 ) + buffer[2];

    //- SPI 통신할 칩 비활성화
    digitalWrite(CS_MCP3208, HIGH);

    return adcValue;
}
```

```
pi@raspberrypi:~/Example/SPI $ gcc -lwiringPi EX_Potentionmeter.c -o POT
pi@raspberrypi:~/Example/SPI $ ls
EX_Potentionmeter.c  POT
```

```
pi@raspberrypi:~/Example/SPI $ ./POT
        Potentionmeter Value = 0
        Potentionmeter Value = 0
        Potentionmeter Value = 225
        Potentionmeter Value = 441
        Potentionmeter Value = 1023
        Potentionmeter Value = 1023
        Potentionmeter Value = 622
        Potentionmeter Value = 263
        Potentionmeter Value = 43
        Potentionmeter Value = 0
        Potentionmeter Value = 169
        Potentionmeter Value = 356
```

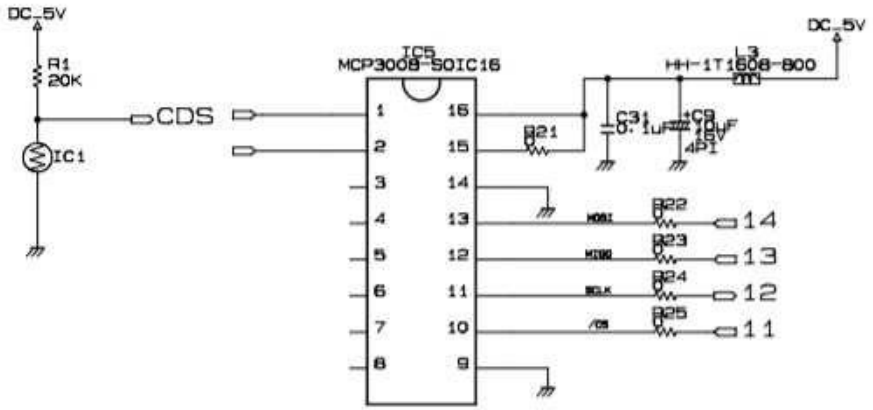
# CDS(1/3)

## 조도(Light)센서

- 광전도효과를 이용하여 주위의 빛의 양을 측정하여 출력해주는 센서
- CDS(Cadmium sulfide): 황화 카드뮴
- 빛이 센서에 조사되면 광전도효과에 의해 내부저항이 감소한다.
- 빛의 세기와 저항 값은 반비례 관계가 성립한다.

| BCM | wPi | Name | Mode | V | Physical | V | Mode | Name | wPi | BCM |
|---|---|---|---|---|---|---|---|---|---|---|
| | | 3.3v | | | 17 \|\| 18 | 0 | IN | GPIO. 5 | 5 | 24 |
| 10 | 12 | MOSI | ALT0 | 0 | 19 \|\| 20 | | | 0v | | |
| 9 | 13 | MISO | ALT0 | 0 | 21 \|\| 22 | 0 | IN | GPIO. 6 | 6 | 25 |
| 11 | 14 | SCLK | ALT0 | 0 | 23 \|\| 24 | 1 | OUT | CE0 | 10 | 8 |
| | | 0v | | | 25 \|\| 26 | 1 | OUT | CE1 | 11 | 7 |
| 0 | 30 | SDA.0 | IN | 0 | 27 \|\| 28 | 0 | IN | SCL.0 | 31 | 1 |
| 5 | 21 | GPIO.21 | OUT | 0 | 29 \|\| 30 | | | 0v | | |
| 6 | 22 | GPIO.22 | IN | 0 | 31 \|\| 32 | 0 | IN | GPIO.26 | 26 | 12 |
| 13 | 23 | GPIO.23 | IN | 0 | 33 \|\| 34 | | | 0v | | |
| 19 | 24 | GPIO.24 | IN | 0 | 35 \|\| 36 | 0 | OUT | GPIO.27 | 27 | 16 |
| 26 | 25 | GPIO.25 | IN | 0 | 37 \|\| 38 | 0 | IN | GPIO.28 | 28 | 20 |
| | | 0v | | | 39 \|\| 40 | 0 | IN | GPIO.29 | 29 | 21 |

Pi 3B



# CDS(2/3)

[ EX_Light.c ]

```c
//- 센서 제어 라이브러리 ----------------------------------------------------------------
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>
#include <errno.h>
#include <string.h>

//- 센서 제어 상수 선언 ----------------------------------------------------------------
#define CHAN_CONFIG_SINGLE        8          //- 싱글 채널 설정 값
#define SPI_SPEED                 1000000    //- 1MHz
#define SPI_CE_CHANNEL            1          //- SPI와 연결된 CE 채널
#define SPI_ADC_LIGHT_CHANNEL     0          //- Light 연결 채널
#define CS_MCP3208                11         //- MCP3208 연결 핀

//- 함수선언----------------------------------------------------------------------------
int spiSetup(void);
int readAnalogData(int adcChannel);

//- 프로그램 실행 함수 ------------------------------------------------------------------
int main(void){

    //- wiringPi 초기화
    if( wiringPiSetup() == -1)
    {
            printf("Setup Fail ~!!");
            return -1;
    }
```

[ EX_Light.c ]

```c
    //- SPI통신 초기화
    spiSetup();

    //- ADC IC MCP3008 초기화
    pinMode(CS_MCP3208, OUTPUT);

    int adcValue = 0;

    //- 센서 읽어서 출력
    while(1)
    {
        //- 센서 값 읽기
        adcValue = readAnalogData(SPI_ADC_LIGHT_CHANNEL);
        //- 입력된 가변저항 값 출력하기
        printf("  Light Value = %dLux₩n", adcValue);
        delay(500);
    }
    return 0;
}


//- SPI통신 라이브러리 초기화------------------------------------------------------------
int spiSetup(void)
{
    if( wiringPiSPISetup (SPI_CE_CHANNEL, SPI_SPEED) == -1 )
    {
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s₩n", strerror (errno));
        return 1;
    }
}
```

# CDS(3/3)

[ EX_Light.c ]

```c
//- 채널 1번에서 측정된 ADC센서 읽어오기 ----------------------------------------
int readAnalogData(int adcChannel)
{
    int adcValue = 0;

    unsigned char buffer[3] = {0};

    //- MCP3008의 8개 채널 중 하나인지 체크
    if(adcChannel<0 || adcChannel>7)  return -1;

    //- SPI 통신 버퍼 설정
    buffer[0] = 0x01;

    buffer[1] = (CHAN_CONFIG_SINGLE+adcChannel) << 4;


    //- SPI 통신할 칩 활성화
    digitalWrite(CS_MCP3208, LOW);


    //- 채널 1에서 측정된 ADC 센서 읽어 오기
    wiringPiSPIDataRW(SPI_CE_CHANNEL, buffer, 3);

    //- 읽어온 값에서 10비트 데이터 추출하기
    adcValue = ( (buffer[1] & 3 ) << 8 ) + buffer[2];

    //- SPI 통신할 칩 비활성화
    digitalWrite(CS_MCP3208, HIGH);

    return adcValue;
}
```

```
pi@raspberrypi:~/Example/SPI $ gcc -lwiringPi EX_Light.c -o LUX
pi@raspberrypi:~/Example/SPI $ ls
EX_Light.c  EX_Potentionmeter.c  LUX  POT
pi@raspberrypi:~/Example/SPI $
```

```
pi@raspberrypi:~/Example/SPI $ ./LUX
        Light Value = 121Lux
        Light Value = 117Lux
        Light Value = 113Lux
        Light Value = 109Lux
        Light Value = 573Lux
        Light Value = 604Lux
        Light Value = 110Lux
        Light Value = 108Lux
        Light Value = 110Lux
```

# 고생하셨습니다

Thank you!